Algorithmes et Pensée Computationnelle

Programmation de base

Le but de cette séance est d'aborder des notions de base en programmation. Au terme de cette séance, vous serez capable de :

- Comprendre les différentes représentations de nombres.
- Définir des variables et comprendre les types de variables existants.
- Utiliser des notions d'algèbre booléenne.
- Écrire des branchements conditionnels.

Les langages qui seront utilisés pour cette séance sont Java et Python. Assurez-vous d'avoir bien installé IntelliJ. Si vous rencontrez des difficultés, n'hésitez pas à vous référer au guide suivant : tutoriel d'installation des outils (macOS) et prise en main de l'environnement de travail ou tutoriel d'installation des outils (Windows).

1 Input / Output

Question 1: (🕓 *5 minutes*) **Output (Java et Python)**
Créez une variable *nom* de type chaîne de caractères contenant votre nom, et une autre *prenom* contenant votre prénom puis affichez : "Bonjour, *prenom nom*".

Question 2: (🕓 *5 minutes*) **Input (Java et Python)**
En vous référant à l'exercice précédent (**Output (Java et Python)**), demandez à l'utilisateur d'entrer son nom et son prénom au lieu d'initialiser vous-même les variables. En Java, utilisez la classe `Scanner` et la méthode `nextLine()`, et en Python, utilisez la fonction `input()`.

2 Typage

Le but de cette partie des travaux pratiques est de comprendre les notions de typage statique et dynamique, ainsi que leur impact sur l'exécution et la gestion des erreurs.

Question 3: (🕓 *5 minutes*) **Les types de variables**

Quelles sont les principaux types de variables (donnez-en trois), comment les déclareriez-vous en Python et en Java ?

Question 4: (🕓 *10 minutes*) **Typage statique et dynamique**

Parmi ces différents programmes, lesquels pourront être exécutés sans problème et lesquels lèveront des erreurs ? Dans le cas où ils génèreraient des erreurs, expliquez la raison de ces erreurs.

Java :

```java
public class Question4 {
    public static void main(String[] args) {
        // Les programmes ci-dessous doivent être exécutés séparément.
        // Programme 1
        int variableOne = 0;
        int variableTwo = 3;
        variableTwo = variableOne;
```

```java

        // Programme 2
        var variableOne = 0;
        var variableTwo = "Hello";
        variableTwo = variableOne;

        // Programme 3
        int variableOne = 0;
        String variableTwo = "Hello";
        variableTwo = variableOne;

        // Programme 4
        var variableOne = 0;
        variableOne = 3.14;

        // Programme 5
        var variableOne = 0;
        String variableTwo = "Hello";
        String variableThree = "World";
        variableTwo = variableThree;

        // Programme 6
        int variableOne = 0;
        variableOne = 3.14;
    }
}
```

Python :

```python
# Programme 7
variable_1 = 0
variable_2 = "Hello"
variable_2 = variable_1

# Programme 8
variable_1 = 0
variable_1 = 3.14
```

⚠ Erreurs fréquentes

Au cas où vous exécuteriez les programmes Java et rencontriez l'erreur suivante : `Exception in thread "main" java.lang.UnsupportedClassVersionError: ... has been compiled by a more recent version of the Java Runtime (class file version 54.0), this version of the Java Runtime only recognizes class file versions up to 52.0`, suivez les étapes ci-dessous :

1. Mettez à jour votre JDK en téléchargeant la version adéquate via le lien suivant : https://www.oracle.com/java/technologies/downloads/.
2. Une fois votre JDK installé, redémarrez IntelliJ
3. Dans la barre de menus, cliquez sur ⋮ > **Edit...**.
4. Dans la fenêtre qui apparaitra (capture ci-dessous), sélectionnez la version du JRE la plus récente.

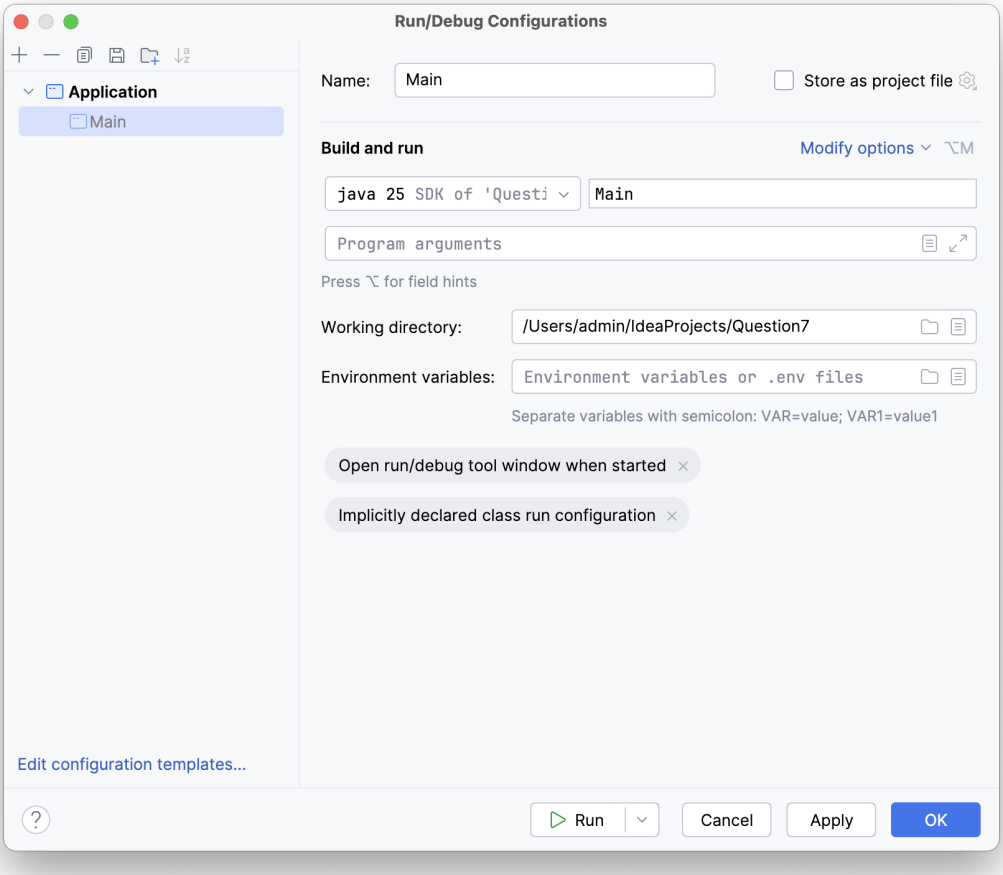


FIGURE 1 – Fenêtre de configuration de l'environnement de développement Java.

3 Opérateurs et conditions booléennes

Le principe d'une valeur booléenne est qu'elle ne puisse contenir que 2 valeurs possibles, soit `True` (`true` en Java), soit `False` (`false` en Java). Il est possible de les définir en leur associant une de ces valeurs d'emblée ou de les obtenir en effectuant une opération booléenne comme par exemple une comparaison. Pour ce faire, il faut utiliser des opérateurs booléens. Voici les plus utilisés : est égal (== en Python et en Java), n'est pas égal (!= en Python et en Java), est strictement plus petit (< en Python et en Java), est plus petit ou égal (<= en Python et en Java), est strictement plus grand (> en Python et en Java), est plus grand ou égal (>= en Python et en Java). Si la condition est satisfaite, on obtiendra `True`, si elle ne l'est pas, on obtiendra `False`. L'utilisation de l'opérateur de négation (`not` en Python et `!` en Java) inversera le résultat.

Question 5: (*7 minutes*)

1. Démontrez l'équivalence suivante :

$$(\neg p \vee q) \wedge (p \vee \neg q) \Leftrightarrow (p \wedge q) \vee (\neg p \wedge \neg q)$$

2. Re-faire cet exercice en utilisant les tables de vérité.

Dans les exercices suivants, vous devrez anticiper la valeur que la console va vous donner (résultat du print).

Question 6: (*5 minutes*) Qu'affiche le programme suivant ?

```
1 a = 3
2 b = 2
```

```
3  c = 6
4  d = c >= b
5  print(a==b or d)
6  print(a==b and d)
7  print(a==b or a==c or False or d)
8  print(a<c and not (4*b==8 and not d))
```

Question 7: (*5 minutes*)

1. Qu'affiche le programme suivant ?

```
1  a = True
2  b = False
3  c = True
4  if a or (b and not c):
5      print("output 1")
6  if b!= c :
7      print("output 2")
8  if a or (not b != (c and a)):
9      print("output 3")
10 else:
11     print("output 4")
```

2. Qu'affiche le programme si l'on remplace la ligne 6 par `elif b != c:` ?

4 Match (Python) et Switch (Java)

L'instruction match en Python et switch en Java a pour objectif de simplifier la logique conditionnelle en sélectionnant un bloc de code à exécuter parmi plusieurs options. Cette structure compare la valeur d'une variable ou d'une expression à une série de cas (case) prédéfinis. Lorsqu'une correspondance est trouvée, le code associé à ce cas est exécuté, offrant ainsi une alternative plus lisible et organisée qu'une longue chaîne de `if-elif-else`.

Il existe toutefois une différence importante entre les deux langages : avec la syntaxe classique de `switch` en Java utilisant les deux-points, l'exécution peut continuer dans les `case` suivants tant qu'aucune instruction `break` n'est rencontrée. Ce comportement est appelé *fall-through*. En Python, `match` n'a pas ce comportement : seul le premier `case` correspondant est exécuté.

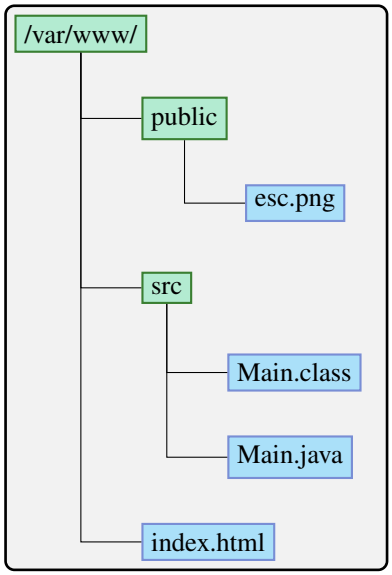
Question 8: (*5 minutes*) Écrire le code Java qui demande à l'utilisateur d'entrer l'étape d'exécution d'une instruction et affiche les étapes qui restent à exécuter (y compris l'étape courante). Si l'utilisateur entre une étape invalide le code affiche `L'étape n'est pas valide.`

Exemple

Si l'utilisateur entre `execute`, le programme va afficher :

```
L'étape execute s'exécute ou va être exécutée.
L'étape store s'exécute ou va être exécutée.
```

Question 9: (*5 minutes*)



Soit l'arborescence à gauche d'un système de fichiers (file-system). Écrivez un programme Python (utilisant `match`) qui affiche **seulement les noms des fichiers** dans un seul dossier spécifié à la ligne de commande. Votre programme doit seulement gérer les arguments suivants :

- `/var/www/./public/..`
- `/var/www/src/..`
- `/var/www/src/`
- `/var/www/public/`

Pour les autres cas, il faut afficher `La saisie n'était pas reconnue`.

Re-faire cet exercice en utilisant `input` pour lire la saisie de l'utilisateur.

5 Type casting

Question 10: (🕓 *5 minutes*) **Conversion des variables (Type casting) (Java et Python)**
Qu'affichent les programmes suivants ?

Python :

```python
nombre_entier = 3
nombre_decimal = float(nombre_entier)
print(nombre_entier)
print(nombre_decimal)
```

Java :

```java
public class Question10 {
    public static void main(String[] args) {
        float nombreDecimal = 3.14f;
        int nombreEntier = (int) nombreDecimal;
        System.out.println(nombreEntier);
        System.out.println(nombreDecimal);
    }
}
```

6 Représentation de nombres entiers

Question 11: (🕓 *5 minutes*) **Entiers non signés**
Sur 8 bits, convertir $113_{(10)}$ en base binaire.

Question 12: (🕓 *5 minutes*) **Entiers signés négatifs**
En utilisant le résultat de la question précédente, convertir sur 8 bits $-113_{(10)}$ en base binaire. Utiliser la représentation **signée** à la diapositive 10 du cours (et non le complément à 1 ou 2).

Question 13: (🕓 *5 minutes*) **Complément à 1**
Écrire $-113_{(10)}$ en complément à 1 sur 8 bits.

Quelle est la différence entre cette méthode et la précédente ?

Question 14: (*5 minutes*) **Complément à 2**
Écrire -$113_{(10)}$ en complément à 2 sur 8 bits, et expliquer l'utilité de cette représentation.

Question 15: (*10 minutes*) **Nombres à virgule flottante (Floating point)**

Voici la représentation en binaire d'un nombre à virgule flottante :

signe	exposant	mantisse
0	10110101	01000001000000000000001

Que vaut cette représentation en base 10 ? Utiliser la représentation des nombres à virgule flottante (avec un biais de 127). Arrondir les résultats intermédiaires et la valeur finale au 3ème chiffre significatif après la virgule.

7 Chaîne de caractères

Question 16: (*10 minutes*)
Soit la chaîne de caractères : `UNIL\nSchool␣of␣Criminal␣Sciences`.

myString=“	U	N	I	L	\n	S	c	h	o	···	”
indices	0	1	2	3	4	5	6	7	8	9 à 31	

Attention

Ici, le caractère ␣ est seulement pour mettre en évidence le caractère espace.

1. Affichez cette chaîne en Python et en Java. Pourquoi est-elle séparée par une nouvelle ligne ?
2. Essayez d'afficher uniquement la chaîne `School of Crimi` en sélectionnant une sous-chaîne de la chaîne initiale.
3. Affichez le troisième caractère de la chaîne en Python et en Java.
4. Afficher la longueur de la chaîne de caractères en Python et en Java.